Державний університет «Житомирська політехніка»

Факультет інформаційно-комп’ютерних технологій
Кафедра комп’ютерної інженерії

ПОЯСНЮВАЛЬНА ЗАПИСКА

до лабораторної роботи № 4
з дисципліни «Програмування»

Модульне програмування, низькорівневі порозрядні операції, бітові маски та командний процес розробки через Pull Request у Git

Індивідуальний варіант 1

Виконав студент 1 курсу
групи КІ-26-1
Безвершук Ростислав

Перевірив Лисенко М. С.
Посада: Старший викладач кафедри комп'ютерної інженерії та штучного інтелекту
Науковий ступінь: Доктор філософії з інженерії програмного забезпечення

Житомир 2026

1 Мета роботи та індивідуальні завдання

Мета роботи - побудувати модульні програми мовою C, дослідити передачу аргументів за значенням і локальний static-стан, виконати операції над бітами 32-бітного регістра та реалізувати пакування телеметрії у 16-бітне слово. Інтеграцію коду організовано за процесом розробки через Pull Request.

Номер у списку - 1. Варіант = ((1 − 1) mod 15) + 1 = 1. Для компіляції використовую GCC із -Wall та -Wextra; для обліку змін - Git і гілку lab04.

1.1 Завдання 1 та розрахунок факторіала

Реалізувати функцію обчислення факторіала. Статична локальна змінна має рахувати сумарну кількість операцій множення за всі виклики. Прототипи розміщую перед main. Обчислювальна factorial приймає n за значенням, а результат, лічильник та статус повертає структурою FactorialResult.

Для $n \geq 2$ факторіал $n! = 2 \cdot 3 \cdot \ldots \cdot n$. Початкове значення результату дорівнює 1, тому $0! = 1$ і $1! = 1$ отримуються без множень. Кількість множень одного виклику дорівнює $n - 1$ при $n \geq 2$ та 0 для $n = 0$ або 1.

Тип uint64_t вміщує 20! = 2432902008176640000, але не 21! = 51090942171709440000. Приймаю цілі n від 0 до 20, перевіряю введення через fgets і strtol та відхиляю зайві символи. Окремо захищаю накопичувальний лічильник від переповнення.

Змінна total оголошена всередині factorial як static і зберігає значення між викликами в одному процесі. Звичайний локальний result створюється для кожного виклику заново. У новому запуску програми total знову починається з 0. Глобальних змінних стану немає.

1.2 Решта індивідуальних завдань

Завдання 2: встановити біт 3, скинути біт 7, інвертувати біт 12 та перевірити стан біта 15 у 32-бітному беззнаковому регістрі. Завдання 3: запакувати код сенсора в біти 0…7, режим у 8…11, тривогу в біт 12 та канал у 13…15; відновити всі поля й перевірити їхню тотожність.

2 Бітові маски та емуляція регістра

Регістр представлено змінною uint32_t. Біт 0 є крайнім правим. Маска M = 1 << N містить одиницю тільки в позиції N. У коді одиниця задається через UINT32_C(1); номери поза діапазоном 0…31 не використовуються для зсуву.

Операція	Вираз	Маска у HEX
Встановити біт 3	reg | M	M = 0x00000008
Скинути біт 7	reg & ~M	M = 0x00000080 ~M = 0xFFFFFF7F
Інвертувати біт 12	reg ^ M	M = 0x00001000
Перевірити біт 15	(reg & M) != 0	M = 0x00008000

Для демонстрації початково встановлено біти 7 та 15: reg = 0x00008080. Такий стан дозволяє побачити справжнє скидання біта 7, а не повторний запис уже нульового біта. Після кожної операції друкую всі 32 біти групами по чотири та HEX із вісьмома цифрами.

Крок	Стан регістра
Початкове значення	0x00008080
set_bit(reg, 3)	0x00008088
clear_bit(reg, 7)	0x00008008
toggle_bit(reg, 12)	0x00009008
check_bit(reg, 15)	1; регістр залишається 0x00009008

XOR між станами до та після знаходить усі змінені біти. Умова ((before ^ after) & ~M) == 0 підтверджує, що поза цільовим бітом зміни відсутні. Перевірка check_bit лише читає значення й повертає 0 або 1.

Це програмна емуляція GPIO: фізичного доступу до апаратного регістра немає. Для реального пристрою Read-Modify-Write може потребувати атомарних засобів або захисту від переривань; volatile саме по собі не забезпечує атомарність.

3 Пакування та розпакування телеметрії

Параметри Telemetry передаю до функції пакування за значенням. До формування пакета перевіряю кожне поле. Пакет має тип uint16_t; проміжні зсуви виконую в uint32_t. Загальна ширина полів: 8 + 4 + 1 + 3 = 16 бітів.

Поле	Діапазон	Біти	Зсув	Маска в пакеті
Код сенсора	0…255	0…7	0	0x00FF
Режим	0…15	8…11	8	0x0F00
Тривога	0…1	12	12	0x1000
Канал	0…7	13…15	13	0xE000

Поля розташовую зсувами вліво та об’єдную через побітове АБО:

```
packet = (sensor & 0xFF)
      | ((mode & 0x0F) << 8)
      | ((alarm & 0x01) << 12)
      | ((channel & 0x07) << 13);
```

Для відновлення зсуваю потрібне поле вправо та залишаю його молодші біти маскою. Наприклад, mode = (packet >> 8) & 0x0F. Маски обмежують ширину полів; некоректні вхідні значення відхиляю до пакування, а не мовчки обрізаю.

Контрольний набір: sensor = 165 = 0xA5, mode = 9, alarm = 1, channel = 5. Компоненти пакета: 0x00A5 | 0x0900 | 0x1000 | 0xA000 = 0xB9A5. Двійковий запис: 1011 1001 1010 0101.

Для 0xB9A5 відновлюються значення 165, 9, 1, 5. Мінімальні поля дають пакет 0x0000, максимальні (255, 15, 1, 7) - 0xFFFF. Функція same_telemetry порівнює саме значення полів, оскільки структура може містити службові проміжки між ними.

4 Лістинг task1.c

```
#include <stdio.h>
#include <stdint.h>
#include <inttypes.h>
#include <stdlib.h>
#include <ctype.h>
#include <errno.h>
#include <string.h>

// Структура дозволяє повернути результат і лічильник без вихідних покажчиків.
typedef struct {
    uint64_t value;
    uint64_t multiplications;
    int valid;
} FactorialResult;

// Прототипи повідомляють компілятору типи функцій до їхнього виклику.
int read_number(unsigned int *n);
FactorialResult factorial(unsigned int n);

int main(void)
{
    unsigned int n;
    puts("Factorial session: enter one integer 0..20 per line.");
    puts("Finish input with EOF (or use a pipe).");
    for (;;) {
        printf("n: ");
        int status = read_number(&n);
        if (status == 0) {
            putchar('\n');
            return 0;
        }
        if (status < 0) {
            fputs("Error: enter one integer from 0 to 20.\n", stderr);
            return 1;
        }
        // Передаємо n за значенням: factorial отримує його копію.
        FactorialResult result = factorial(n);
        if (!result.valid) {
            fputs("Error: factorial or counter is out of range.\n", stderr);
            return 1;
        }
        printf("%u! = %" PRIu64 "\n", n, result.value);
        printf("Total multiplications: %" PRIu64 "\n",
               result.multiplications);
        printf("n in main is still: %u\n", n);
    }
}
```

4 Лістинг task1.c продовження

```
int read_number(unsigned int *n)
{
    char line[128];
    if (fgets(line, sizeof(line), stdin) == NULL) {
        return feof(stdin) ? 0 : -1;
    }
    if (strchr(line, '\n') == NULL && !feof(stdin)) {
        return -1;  // Не приймаємо частково зчитаний довгий рядок.
    }
    char *end;
    errno = 0;
    long value = strtol(line, &end, 10);
    if (end == line || errno == ERANGE) {
        return -1;
    }
    while (isspace((unsigned char)*end)) {
        ++end;
    }
    if (*end != '\0' || value < 0 || value > 20) {
        return -1;
    }
    *n = (unsigned int)value;
    return 1;
}

FactorialResult factorial(unsigned int n)
{
    // Ініціалізується один раз і зберігається між викликами в цій сесії.
    static uint64_t total = 0;
    FactorialResult result = {1, total, 0};
    if (n > 20) {
        return result;  // 21! уже не вміщується в uint64_t.
    }
    unsigned int needed = n > 1 ? n - 1 : 0;
    if (total > UINT64_MAX - needed) {
        return result;  // Захищаємо і накопичувальний лічильник.
    }
    // 0! = 1 і 1! = 1: для них цикл не виконується.
    for (unsigned int i = 2; i <= n; ++i) {
        result.value *= i;
        ++total;  // Рахуємо саме виконане множення, а не виклик функції.
    }
    result.multiplications = total;
    result.valid = 1;
    return result;
}
```

5 Лістинг task2.c

```
#include <stdio.h>
#include <stdint.h>
#include <inttypes.h>

uint32_t set_bit(uint32_t reg, unsigned int n);
uint32_t clear_bit(uint32_t reg, unsigned int n);
uint32_t toggle_bit(uint32_t reg, unsigned int n);
int check_bit(uint32_t reg, unsigned int n);
void print_binary(uint32_t reg);
void show_register(const char *label, uint32_t reg);
int other_bits_preserved(uint32_t before, uint32_t after, unsigned int n);

int main(void)
{
   // Біти 7 і 15 початково увімкнені, щоб скидання біта 7 було видимим.
   uint32_t reg = UINT32_C(0x00008080);
   uint32_t before = reg;
   int ok = 1;
   show_register("Before set bit 3", reg);
   reg = set_bit(reg, 3);
   show_register("After set bit 3", reg);
   ok &= other_bits_preserved(before, reg, 3);

   before = reg;
   show_register("Before clear bit 7", reg);
   reg = clear_bit(reg, 7);
   show_register("After clear bit 7", reg);
   ok &= other_bits_preserved(before, reg, 7);

   before = reg;
   show_register("Before toggle bit 12", reg);
   reg = toggle_bit(reg, 12);
   show_register("After toggle bit 12", reg);
   ok &= other_bits_preserved(before, reg, 12);

   before = reg;
   printf("Bit 15 state: %d\n", check_bit(reg, 15));
   printf("Check did not change register: %s\n", before == reg ? "PASS" : "FAIL");
   printf("All other-bit checks: %s\n", ok ? "PASS" : "FAIL");
   return ok ? 0 : 1;
}
```

5 Лістинг task2.c продовження

```
uint32_t set_bit(uint32_t reg, unsigned int n)
{
   if (n >= 32) return reg;  // Не можна зсувати на 32 або більше бітів.
   return reg | (UINT32_C(1) << n);  // АБО примусово встановлює цільовий біт.
}

uint32_t clear_bit(uint32_t reg, unsigned int n)
{
   if (n >= 32) return reg;
   return reg & ~(UINT32_C(1) << n);  // І з інверсією маски скидає біт.
}

uint32_t toggle_bit(uint32_t reg, unsigned int n)
{
   if (n >= 32) return reg;
   return reg ^ (UINT32_C(1) << n);  // XOR змінює 0 на 1 або 1 на 0.
}

int check_bit(uint32_t reg, unsigned int n)
{
   if (n >= 32) return 0;
   return (reg & (UINT32_C(1) << n)) != 0;  // Повертаємо саме 0 або 1.
}

void print_binary(uint32_t reg)
{
   for (int i = 31; i >= 0; --i) {
      putchar(check_bit(reg, (unsigned int)i) ? '1' : '0');
      if (i % 4 == 0 && i != 0) putchar(' ');  // Групуємо по 4 біти.
   }
   putchar('\n');
}

void show_register(const char *label, uint32_t reg)
{
   printf("%-22s 0x%08" PRIX32 "  ", label, reg);
   print_binary(reg);
}

int other_bits_preserved(uint32_t before, uint32_t after, unsigned int n)
{
   if (n >= 32) return 0;
   uint32_t mask = UINT32_C(1) << n;
   // XOR знаходить змінені біти; ~mask прибирає з перевірки цільовий біт.
   int ok = ((before ^ after) & ~mask) == 0;
   printf("Other bits preserved: %s\n", ok ? "PASS" : "FAIL");
   return ok;
}
```

6 Лістинг task3.c

```
#include <stdio.h>
#include <stdint.h>
#include <inttypes.h>
#include <stdlib.h>
#include <ctype.h>
#include <errno.h>
#include <string.h>

typedef struct {
    unsigned int sensor;
    unsigned int mode;
    unsigned int alarm;
    unsigned int channel;
} Telemetry;

int read_telemetry(Telemetry *data);
uint16_t pack_telemetry(Telemetry data);
Telemetry unpack_telemetry(uint16_t packet);
int same_telemetry(Telemetry a, Telemetry b);

int main(void)
{
    Telemetry input;
    printf("Enter sensor(0..255) mode(0..15) alarm(0..1) channel(0..7): ");
    if (!read_telemetry(&input)) {
        fputs("Error: expected four integers in ranges 0..255 0..15 0..1 0..7.\n",
              stderr);
        return 1;
    }
    printf("Original: sensor=%u mode=%u alarm=%u channel=%u\n",
           input.sensor, input.mode, input.alarm, input.channel);
    uint16_t packet = pack_telemetry(input);
    printf("Packet HEX: 0x%04" PRIX16 "\n", packet);
    Telemetry restored = unpack_telemetry(packet);
    printf("Restored: sensor=%u mode=%u alarm=%u channel=%u\n",
           restored.sensor, restored.mode, restored.alarm, restored.channel);
    int ok = same_telemetry(input, restored);
    printf("Round-trip: %s\n", ok ? "PASS" : "FAIL");
    return ok ? 0 : 1;
}
```

6 Лістинг task3.c продовження

```
int read_telemetry(Telemetry *data)
{
   char line[128];
   if (fgets(line, sizeof(line), stdin) == NULL) return 0;
   if (strchr(line, '\n') == NULL && !feof(stdin)) return 0;
   const long limits[4] = {255, 15, 1, 7};
   unsigned int values[4];
   char *cursor = line;
   for (int i = 0; i < 4; ++i) {
      char *end;
      errno = 0;
      long value = strtol(cursor, &end, 10);
      if (end == cursor || errno == ERANGE || value < 0 || value > limits[i]) {
         return 0;
      }
      if (*end != '\0' && !isspace((unsigned char)*end)) return 0;
      values[i] = (unsigned int)value;
      cursor = end;
   }
   while (isspace((unsigned char)*cursor)) ++cursor;
   if (*cursor != '\0') return 0;  // Зайвих значень чи символів бути не повинно.
   data->sensor = values[0];
   data->mode = values[1];
   data->alarm = values[2];
   data->channel = values[3];
   return 1;
}

uint16_t pack_telemetry(Telemetry data)
{
   // Приймаємо перевірені значення. Маски задають ширину кожного поля.
   // Зсуви виконуємо у uint32_t; готове 16-бітне слово повертаємо як uint16_t.
   uint32_t packet = ((uint32_t)data.sensor & UINT32_C(0xFF))
      | (((uint32_t)data.mode & UINT32_C(0x0F)) << 8)
      | (((uint32_t)data.alarm & UINT32_C(0x01)) << 12)
      | (((uint32_t)data.channel & UINT32_C(0x07)) << 13);
   return (uint16_t)packet;
}

Telemetry unpack_telemetry(uint16_t packet)
{
   Telemetry data;
   // Переносимо потрібне поле вправо та відкидаємо решту бітів маскою.
   data.sensor = (unsigned int)((uint32_t)packet & UINT32_C(0xFF));
   data.mode = (unsigned int)(((uint32_t)packet >> 8) & UINT32_C(0x0F));
   data.alarm = (unsigned int)(((uint32_t)packet >> 12) & UINT32_C(0x01));
   data.channel = (unsigned int)(((uint32_t)packet >> 13) & UINT32_C(0x07));
   return data;
}

int same_telemetry(Telemetry a, Telemetry b)
{
   // Порівнюємо значення полів, а не байти структури з можливими відступами.
   return a.sensor == b.sensor && a.mode == b.mode
      && a.alarm == b.alarm && a.channel == b.channel;
}
```

7 Тестування факторіала та static-стану

Для першого контрольного запуску передаю 3, 5, 0 і 20 одним потоком введення до одного процесу task1. Лічильник total накопичується між викликами; число n в main залишається таким самим після обчислення.

n	n!	Множень у виклику	Накопичено
3	6	2	2
5	120	4	6
0	1	0	6
20	2432902008176640000	19	25



Рисунок 1 - Факторіали та накопичувальна кількість множень

Окремо перевіряю n = −1 і n = 21. Обидва значення відхиляються повідомленням Error: enter one integer from 0 to 20. Код завершення перевіряю через echo $?; він дорівнює 1.

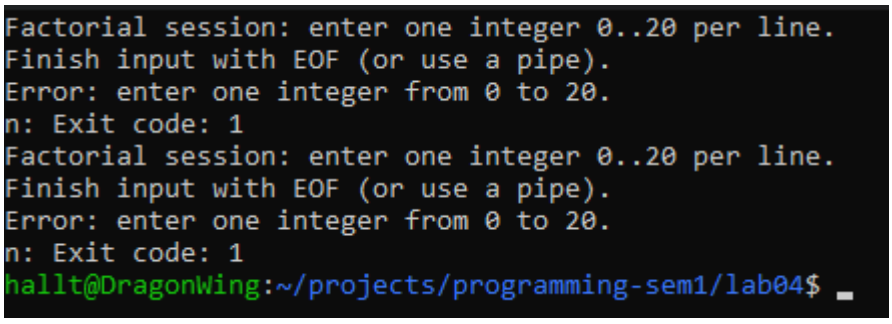


Рисунок 2 - Відхилення від’ємного аргументу та переповнення факторіала

8 Тестування регістра та звичайного пакета

У task2 отримую послідовність 0x00008080 → 0x00008088 → 0x00008008 → 0x00009008. Для кожної зміни друкується Other bits preserved: PASS; підсумок All other-bit checks також має значення PASS. Біт 15 дорівнює 1, а його перевірка не змінює регістр.

```
Before set bit 3       0x00008080  0000 0000 0000 0000 1000 0000 1000 0000
After set bit 3        0x00008088  0000 0000 0000 0000 1000 0000 1000 1000
Other bits preserved: PASS
Before clear bit 7     0x00008088  0000 0000 0000 0000 1000 0000 1000 1000
After clear bit 7      0x00008008  0000 0000 0000 0000 1000 0000 0000 1000
Other bits preserved: PASS
Before toggle bit 12   0x00008008  0000 0000 0000 0000 1000 0000 0000 1000
After toggle bit 12    0x00009008  0000 0000 0000 0000 1001 0000 0000 1000
Other bits preserved: PASS
Bit 15 state: 1
Check did not change register: PASS
All other-bit checks: PASS
```

Рисунок 3 - Двійкові стани регістра до та після бітових операцій

Для введення 165 9 1 5 у task3 сформовано Packet HEX: 0xB9A5. Після розпакування початкові й відновлені значення збігаються, що підтверджує Round-trip: PASS.

```
Enter sensor(0..255) mode(0..15) alarm(0..1) channel(0..7): Original: sensor=165 mode=9 alarm=1 channel=5
Packet HEX: 0xB9A5
Restored: sensor=165 mode=9 alarm=1 channel=5
Round-trip: PASS
hallt@DragonWing:~/projects/programming-sem1/lab04$
```

Рисунок 4 - Пакування та відновлення контрольної телеметрії

9 Граничні значення та некоректне введення

Перевіряю найменший набір 0 0 0 0 та найбільший 255 15 1 7. Отримую відповідно 0x0000 й 0xFFFF. В обох випадках відновлені поля збігаються з початковими, а Round-trip має значення PASS.

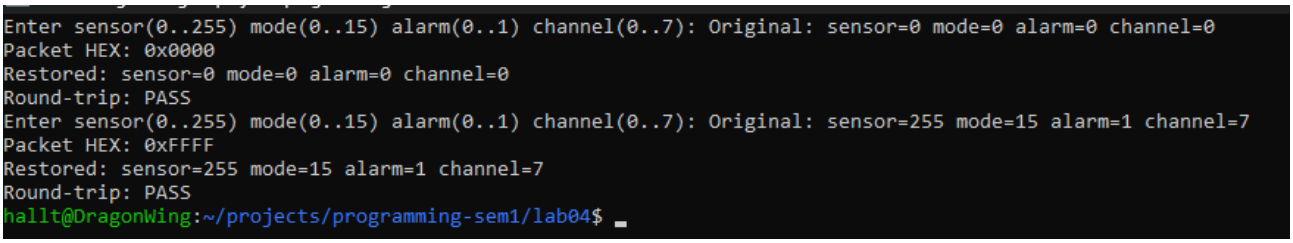


Рисунок 5 - Мінімальні та максимальні значення всіх полів пакета

Для кожного поля окремо перевіряю вихід за верхню межу. У всіх чотирьох випадках task3 повідомляє про помилку, не формує пакет і завершується з кодом 1.

Введення	Порушення
256 0 0 0	sensor > 255
0 16 0 0	mode > 15
0 0 2 0	alarm > 1
0 0 0 8	channel > 7

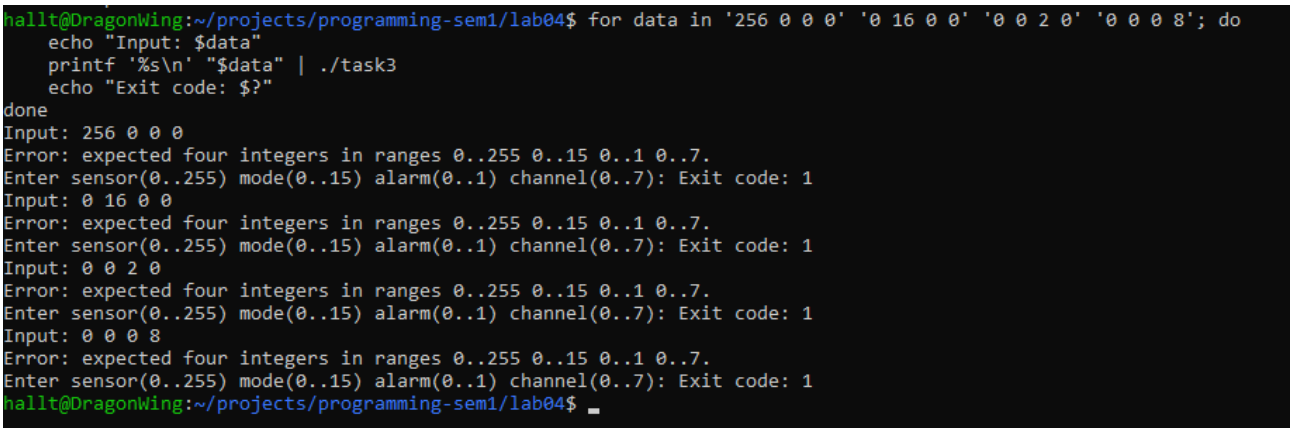


Рисунок 6 - Відхилення параметрів поза допустимими діапазонами

10 Контроль версій та висновки

Процес роботи з Git передбачає створення гілки lab04 від main, окремі коміти для task1.c, task2.c, task3.c та Report_Lab04.pdf і відправку lab04 на GitHub. Зміни включаються до main через Pull Request із назвою Lab 4: Functions, Bitwise Operations and Register Emulation. Після перевірки PR зливається зі статусом Merged, а локальна main синхронізується командою git pull origin main.

Для варіанта 1 реалізовано функцію факторіала з передаванням аргументу за значенням. Результат та статистика повертаються структурою, а локальний static-лічильник зберігає сумарну кількість фактично виконаних множень. Допустимий діапазон 0…20 захищає uint64_t від переповнення факторіала. Функція введення працює з покажчиком лише для запису прочитаного числа; обчислювальна factorial вихідних покажчиків не використовує.

Для регістра застосовано OR, AND з інверсією маски, XOR та AND із маскою. Контроль XOR-різниці підтвердив збереження всіх нецільових бітів. При роботі з телеметрією чотири поля без перетинів займають усі 16 бітів, а зворотні зсуви й маски відновлюють їх без втрат.

Усі три вихідні файли скомпільовано GCC із -std=c11 -Wall -Wextra -Wpedantic -Werror без попереджень. Під час додаткової перевірки факторіали 0…20 звірено з незалежним обчисленням; бітові операції перевірено для всіх 32 позицій на п’яти станах регістра; пакування та розпакування перевірено для всіх 65 536 можливих слів. Також перевірено 20 випадків некоректного введення.

Мету обчислювальної частини роботи досягнуто: застосовано прототипи й функціональну декомпозицію, досліджено static-стан, беззнакові зсуви та бітові маски. Контрольні результати узгоджуються з ручними розрахунками. Підсумковий комплект для захисту містить три програмні файли, пояснювальну записку зі скріншотами та відповідний Pull Request у репозиторії.